Question Answering System with BERT

Course: Natural Language Processing Applications **Due Date:** [17-Dec-2025 23:59]

1. Introduction

Extractive Question Answering (QA) is a core task in Natural Language Processing that involves finding an answer to a question from within a given text passage (or "context"). The answer is a direct "span" of text from the context. In recent years, pre-trained transformer models like BERT (Bidirectional Encoder Representations from Transformers) have achieved state-of-the-art results on this task.

In this assignment, you will leverage the Hugging Face `transformers` library to fine-tune a pre-trained BERT model on a question-answering dataset. You will then build a simple web application to interact with your fine-tuned model.

2. Learning Objectives

Upon successful completion of this assignment, you will be able to:

- **Process** a given context/passage and a related question into a format suitable for BERT.
- **Fine-tune** a pre-trained BERT model on an extractive QA dataset.
- **Achieve** reasonable accuracy on a reading comprehension task and understand how to evaluate it.
- **Build** a simple web application that takes a context and a natural language question, and provides a textual answer.

3. Recommendations

You can use **hotpotqa/hotpot_qa** dataset from huggingface.

- **hotpot_qa:** HotpotQA is a new dataset with 113k Wikipedia-based question-answer pairs

- **1 Mark** for using BITS CSIS Virtual Labs, attach a few screenshots as proof of execution within the CSIS Labs.

4. Assignment Tasks

This assignment is broken into three main parts.

Part 1: Data Preprocessing [4 Marks]

This is the most critical step. A standard BERT model cannot directly accept a raw context and question. You must pre-process the data.

1. Import Dataset and extract features and labels.
2. Perform EDA and analyse question types and answer distributions

3. Print answers for 5 context and questions.
4. Build a simple web page which takes context and question as input and provide answers. (in case this is not done, context and question can be provided in command line or jupyter notebook)
5. Ensure the code (or all cells in jupyter notebook) has been executed and displayed, and should not have any errors.

Part 2: Model Fine-Tuning [5 Marks]

1. Load pre-trained BERT model and Fine tune BERT model with this dataset.
2. Experiment with hyperparameters
3. Your model should take question and context as input and provide answers.

Part 3: Simple Web Application [2 Marks]

Build a simple web application (using **Flask** or **FastAPI** is recommended) that serves your fine-tuned model.

Below is the only guidelines, you are free to use alternative approaches to build an Application.

1. **Backend (e.g., `app.py` with Flask):**
    - Load your *fine-tuned model* and *tokenizer* from the directory you saved in Part 2.
    - Create a single API endpoint (e.g., `/predict`) that accepts a JSON POST request containing `{"context": "...", "question": "..."}`.
    - Inside this endpoint, use your model to run inference:
        - Preprocess the context and question (tokenize, format as `[CLS]... [SEP]...[SEP]`).
        - Pass the inputs to the model.
        - Get the output `start_logits` and `end_logits`.
        - Find the `(start_index, end_index)` pair with the highest logit sum.
        - Decode the tokens from `start_index` to `end_index` back into a string.
    - Return the predicted answer string as a JSON response (e.g., `{"answer": "..."}`).
2. **Frontend (e.g., `index.html`):**
    - Create a simple HTML page with:
        - A large `<textarea>` for the context.
        - An `<input type="text">` for the question.
        - A `<button>` to submit.
        - A `<div>` or `<p>` tag to display the model's answer.

- Write basic JavaScript (using `fetch`) to:
    - Get the text from the context and question fields.
    - Send this data to your `/predict` backend endpoint.
    - Wait for the JSON response and display the answer in the answer `<div>`.

Part 4: Design and Research [3 Marks]

- Develop a comprehensive design document that synthesizes findings from current academic literature to outline your theoretical approach.
- Discuss how the choice of various word embedding techniques will affect your model performance.
- Analyse recent advances in transformer architectures, attention mechanisms, and model scaling as documented in academic papers.
- Discuss potential enhancements such as knowledge integration, cross-lingual adaptation, and robustness improvements, supporting each proposal with citations to relevant research publications and empirical studies.
- Review contemporary methodologies and compile relevant references.

5. Submission

Please submit a ZIP file containing:

PART - A

- A well-documented code Jupyter Notebook of the code and approach. Ensure all cells are executable.
- All cells should have output and should not error-out when run locally.
- Screenshots of using CSIS Virtual Labs
- Screenshots of test cases with clear input and outputs.

PART – B

- Submit a design document of the approach taken. Design document should also contain improvements which can be made to the solution, and limitations of solution.
- Survey modern approaches and document references and discuss how that can be used to finetune your models.